10/9/24. EXP 6 – HAMMING CODE

AIM:

Write a program to implement error detection and correction using Hamming code concepts. Make a test run to the input data stream and verify error correction feature.

Sender Code:

```
m = int(input("Enter the number of data bits"))
data = input("Enter data bits")
if len(data) != m or any(bit not in "01" for bit in data):
    print("invalid data")
    exit()
r=0
while (2**r) < (m+r+1):
    r+=1
print("Number of parity bits :", r)
n = m+r
hamming [0]*(n+1)
j=0
for i in range(1,n+1):
    if (i == (2**(j-1))):
        hamming[i] = data[j]
        j+=1
for p in range(r):
    parity_position = 2**p
    parity = 0
```

```
for i in range (1, n+1):
    if i & parity-position:
        parity ^= int(Hamming[i])
hamming[parity-position] = str(parity).
hamming_code = ""
for i in range (n, 0, -1):
    hamming_code += hamming[i].
print("Original data :", data).
print("Hamming code:", hamming_code)
```

Receiver Code :

```
received = input("Enter revived Hamming
                  code"))
if any (bit not in "01" for bit in received):
    print ("Invalid Hamming Code")
    exit().
n = len.(received).
r = 0.
while (2**r) <= n:
    r += 1
print("Number of parity bits.", r-1).
Hamming. = ['0'] + list (receive[::-1]).
error-position = 0.
for p in range(p):
    parity-position = 2**p.
    parity = 0.
for i in range (1, n+1).
    if i in parity-position
        parity ^= int (Hamming[i])
if parity != 0.
    error_position += parity-position.
if error_position == 0.
    print ("No Error Detected").
else.
    print ("error detected ! ")
    print ("error position from right", error_position)
    if hamming[error-position] == '0':
        hamming [error_position] = '1';
    else:
        hamming [error_position] = '0';
corrected_code = "".
for i in range (n, 0, -1)
    corrected_code += Hamming [i].
print (corrected_code)
```

RESULT:

Error detection + error correction using Hamming code are learned and executed succesfully